Report *"Team PK"*

Leonardo Suriano
suriano.2264349@studenti.uniroma1.it
Mariana Santos
dossantos2238828@studenti.uniroma1.it
Riccardo Pugliese
pugliese.2264350@studenti.uniroma1.it

1. Introduction

In this report we are going to explain shortly our models, giving some insight in our work and tackling the approaches we tried, what worked and what did not and which of our models, is our final.

2. Methods

Our methodology is centered on a robust feature engineering pipeline designed to convert nested JSONL battle logs into a structured feature matrix suitable for our analysis. This pipeline serves as the foundation for all subsequent modeling.

To predict the result of the Pokémon battles, we developed and compared three distinct modeling strategies. We began developing a simple, interpretable baseline to then apply logistic regression on a small number of features. The result of these attempts is the model (1). After that, we started creating new features (though most of them have been then discarded) to maximize information gain. With those features we have then implemented an ensemble made of Adaboost, XGBoost and Logistic Regression (model 3).

2.1. Feature Engineering

In each model there is a different features builder, that is building the features needed by the model to better predict the outcome. This process transformed each battle into a single row in a feature table, resulting in the matrices X (features), y (target), and $Xtest$ (test features). For further details about the variables we used please check directly the code on GitHub.

2.2. Approach 1: Interpretable 5-Feature Logic Regression

In our first approach, we implemented a minimal and interpretable model, building a compact set of only five features. A standard LogisticRegression model from scikit-learn, combined with a StandardScaler, was then trained on these five features, tuning hyperparameters (like C, $penalty$, $solver$) using GridSearchCV. We used, to measure the performance of our model, a 5-fold cross validation. When it comes to the simplicity/accuracy trade off, this model is the best in terms of simplicity, having few features.

2.3. Approach 2: High-Dimensional Regularized Logistic Regressions

In our first approach, we implemented a minimal and interpretable model, building a compact set of only five features. A standard LogisticRegression model from scikit-learn, combined with a $StandardScaler$, was then trained on these five features, tuning hyperparameters (like C, $penalty$, $solver$) using GridSearchCV. We used, to measure the performance of our model, a 5-fold cross validation. When it comes to the simplicity/accuracy trade off, this model is the best in terms of simplicity, having few features.

2.4. Approach 3: Soft-Voting Ensemble

Our final approach aimed to capture what simple model such the logistic regression may have been missing out. We built a soft-voting ensemble using scikit-learn's $VotingClassifier$. This model combined the predictions of three independently tuned base learners: 1. Logistic Regression: A regularized linear model (similar to Approach 2). 2. AdaBoost: An adaptive boosting model using shallow decision trees to focus on hard-to-classify samples. 3. XGBoost: A powerful gradient boosting classifier known for its performance on structured data. Each base model was tuned using GridSearchCV or RandomizedSearchCV. The final ensemble weights were proportional to the cross-validation accuracy of each component (using 5 folds), allowing the more accurate models to have a greater influence on the final prediction. This model is the one that, in our trade-off simplicity/accuracy, is it maximizing accuracy.

3. Experiments and Results

We evaluated our three proposed models using two metrics: (1) the mean 5-fold stratified cross-validation (CV) accuracy on the training set, and (2) the final score on the public Kaggle leaderboard. All models were trained and tuned using scikit-learn's GridSearchCV or RandomizedSearchCV to find the optimal hyperparameters. Perfomance wise, our results are summarized below: • Approach 1 (5-Feature LR): This baseline model achieved a 5-fold CV local accuracy of 0.8426, and a final Kaggle score on the public leaderboard of 0.8253 • Approach 2 (High-Dim LR): The high-dimensional linear model improved on the baseline, achieving a 5-fold CV local accuracy of 0.8512 and a Kaggle score on the public leaderboard of 0.844 • Approach 3 (Ensemble): Our final ensemble model yielded the best performance, achieving a 5-fold CV local accuracy of 0.8538 and a final Kaggle score on the public leaderboard of 0.846.

3.1. Analysis: What Worked and What Didn't

What worked well: The ensemble model (Approach 3) was our most successful strategy, and the techniques we

implemented turned out to be working fine, managing to build a robust model that generalized well to the unseen test data. What didn't: Everything we tried, with some AI help, worked fine, but the approach that didn't satisfy us fully was the second one, because, despite having more than 6 times the features of our first model, performed, locally, only slightly better than the first one. We don't think the small jump in accuracy justifies the extra computation needed for building the features. This suggests that simply adding more features to a linear model yield diminishing returns, as it struggles to model the complex, conditional logic inherent in Pokémon (e.g., "Attack X is good, unless the opponent has Type Y"). This limitation justified our move to the more complex ensemble, that returns better results (but still in our view, given the only marginal accuracy boost, we don't think the extra computational power needed to run the more complex model and build the additional features is justified).

4. Final Model Strategy

Based on its superior performance on both the cross-validation and the final leaderboard, our final submitted model was the Soft-Voting Ensemble (Approach 3). It is important, however, to highlight that this choice has been solely driven by the competition we are competing in. In the competition, what matters is the final performance, and our third model is the best one accuracy-wise. If the competition was more focused on other aspects, like readability, simplicity and/or finding the sweet spot in the trade off between simplicity and accuracy, we would have submitted as final model our first model (the simple logistic regression with 5 variables).

5. Discaimers and final thanks

Parts of our code (in particular some comments in the code, the iterative feature search and minor implementation details) may have been drafted or refined with the help of AI-based tools. The use of AI however was strictly limited to these aspects and all core ideas, modeling choices, and logical structures implemented in the code and in the models were entirely conceived and designed by the members of the group, without external intellectual contribution, relying solely on online documentation plus our own knowledge and the insights provided by the course lectures. Talking about lectures, we want to thank the professor Indro Spinelli and his assistants Leonardo Rocci and Simone Facchiano. All of them have given all us students not only a big help, but also the knowledge required to participate in such competition. Moreover, their lessons have been interesting and stimulating, something that, unfortunately, it is not so easy to find in Italian university. We really appreciate your work for us and the effort that, day by day, you are dedicating to making the lessons interesting for us and, more generally, the course more interesting.